

Autoregressive Generation

How language models generate text one token at a time — and what can go wrong

25 min

Duration

B2 – Understand

Bloom's

L2 – Foundational

Level

Topic 1 (Prob.)

Builds on

Learning Objectives

By the end of this session, you will be able to:

B2 – Explain

The autoregressive loop: how one token prediction feeds the next

B2 – Describe

The three mechanisms that stop generation: EOS, max_tokens, stop sequences

B2 – Identify

Failure modes of autoregression: repetition loops and hallucination

B3 – Distinguish

The three hallucination types: intrinsic, extrinsic, and factual

B3 – Explain

Why high model confidence does not imply factual correctness

B4 – Evaluate

Common misconceptions about how generation actually works

Session Roadmap

6 sub-topics — loop mechanics to failure modes and misconceptions

2.1 Recap — From Token Prediction to Generation

Bridge from Topic 1

2.2 The Autoregressive Loop

Core mechanism

2.3 Stopping Conditions

EOS · max_tokens · stop sequences

2.4 Failure Modes — Repetition & Degenerate Outputs

Greedy loops

2.5 Hallucination — Deep Treatment

Types · causes · fixes

2.6 Common Misconceptions — Thoroughly Cleared

3 critical corrections

2.1

Recap — From Token Prediction to Generation

Topic 1 gave us the mechanism. Topic 2 shows what happens when we repeat it.

From Token Prediction to Generation

One mechanism, applied repeatedly

Topic 1 established:

- A language model outputs a probability distribution over all $|V|$ tokens
- Softmax converts logits to valid probabilities summing to 1
- A sampling strategy selects ONE token from that distribution



*Topic 2
asks:*

Topic 2 answers:

What happens to the sampled token?

It is appended to the context, the extended sequence is fed back in, and the process repeats — one token at a time until generation stops.

Autoregression = the same softmax mechanism from Topic 1, applied in a loop. No new math. One new structure.

Chain rule connection:

$P(x_1, x_2, \dots, x_n) = P(x_1) * P(x_2|x_1) * P(x_3|x_1, x_2) * \dots * P(x_n|x_1, \dots, x_{n-1})$ -- Each factor is one forward pass.

2.2

The Autoregressive Loop

Five stages, cycling until generation stops

The Autoregressive Loop

Each forward pass produces exactly one new token — no more, no less



Worked Example — generating the response to: 'The capital of France is'

Pass 1	Context: "The capital of France is"	→ Paris (P=0.72)
Pass 2	Context: "The capital of France is Paris"	→ <EOS> (P=0.91)

Assignment A2 — The Autoregressive Loop

Make the loop tangible — count forward passes with your own hands

WHAT · WHY · EXPECTED OUTCOME

WHAT

Call the API with `stream=True`. Log each token as it arrives in the stream.

Count the total number of tokens received and note the API-reported usage token count.

Verify: streamed token count = reported output token count.

WHY

To make autoregression tangible — you verify with your own hands that generation is sequential, one forward pass per token.

Connects the loop diagram on the previous slide to a real API response you can observe live.

EXPECTED OUTCOME

A streamed token log showing tokens arriving one at a time.

A one-sentence answer to: why does a 200-token response cost exactly twice a 100-token response?

If you can write that sentence correctly, you understand autoregression.

`stream=True, log each chunk.choices[0].delta.content` · Submit notebook before mentor session

2.3

Stopping Conditions

Three mechanisms that halt the loop — all controllable via the API

Stopping Conditions

The loop runs until one of three conditions is met — in this priority order

Priority: ① EOS token sampled → ② Stop sequence matched → ③ max_tokens ceiling hit

① EOS Token — End of Sequence

What it is:

A special token in the vocabulary (e.g. <endoftext> in GPT-4). The model learns during training to predict it when a response is naturally complete.

How it works:

When the sampling step draws the EOS token, generation halts immediately. The EOS token itself is not returned in the output.

finish_reason: stop

Linguistic — model decides

② Stop Sequences — User Defined

What it is:

Strings specified by the caller that trigger halt the moment they appear in the output. E.g. double newline, ### markers, or XML closing tags.

How it works:

After each token is appended, the output is scanned for any stop sequence. On match, generation halts and the stop string is excluded from the returned text.

finish_reason: stop

Structural — caller decides

③ max_tokens — Hard Ceiling

What it is:

A hard limit on output tokens set by the API caller. If EOS and stop sequences have not triggered, this cuts generation off at the specified count.

How it works:

Token counter increments each step. When it reaches max_tokens, generation halts regardless of whether the output is linguistically complete.

finish_reason: length

Economic — cost and safety

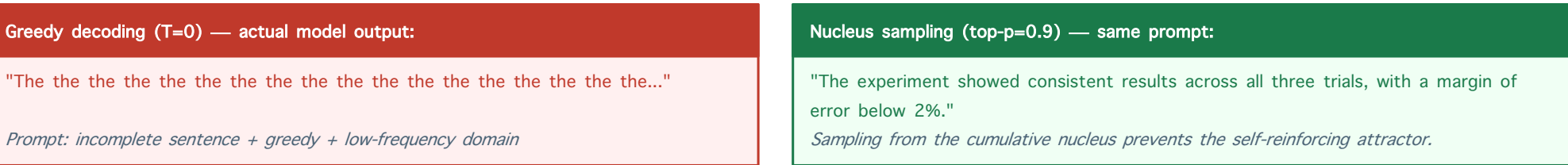
2.4

Failure Modes — Repetition & Degenerate Outputs

What happens when the loop reinforces itself

Greedy decoding's most common pathology — and why nucleus sampling was invented to fix it

Under greedy decoding, once a high-probability phrase is generated, it shifts the context such that the same phrase becomes high-probability again. The probability landscape tilts toward what has already been said. The loop self-reinforces and the model cannot escape it.



Prof. Sashikumaar Ganesan | IISc Bangalore

2.5

Hallucination — Deep Treatment

Plausible is not True. The most important failure mode in production LLMs.

What Hallucination Is — Precisely

Plausibility and truth are different things. The model was trained to be plausible, not true.

Hallucination: the model generates a token that is plausible given the local context — it has high softmax probability — but is factually incorrect when evaluated against the real world.

What the model optimises for

- Predict the next token with high probability
- Match statistical patterns in training data
- Produce text that looks like human writing
- Maximise $P(\text{token} \mid \text{context})$

GAP

What the model does NOT optimise for

- Verify facts against external knowledge
- Check consistency with real-world events
- Know what it does not know
- Abstain when uncertain

Root cause: The model has no world model, no external fact store, and no verification step. It asks only: what token is statistically likely here given this context?

Three Types of Hallucination

Named, defined, and distinguished — each has a different cause and different fix

TYPE 1

Intrinsic Hallucination

The output directly contradicts information present in the input context.

Example:

Document: The meeting was on Monday.
Model: The meeting scheduled for Tuesday was...

Mitigation:

Shorter clearer context; instruct faithfulness; faithfulness checker.

TYPE 2

Extrinsic Hallucination

The output adds information that cannot be verified or contradicted by the input context.

Example:

Document: John worked at the company for years.
Model: John, who holds an MBA from Stanford, worked...

Mitigation:

Instruct to respond only from context; use citations; RAG with strict grounding.

TYPE 3

Factual Hallucination

The output contradicts established real-world knowledge not present in any context.

Example:

The Eiffel Tower was designed by Claude Monet and completed in 1901.

Mitigation:

RAG to ground in retrieved facts; RLHF to reduce confident wrong answers; human review.

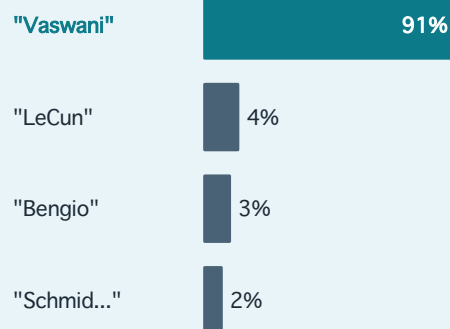
High Confidence Does Not Mean Correct

The most dangerous property of hallucination — it passes casual human review

A model can produce a hallucinated claim with $P(\text{token}) > 0.95$ — the softmax is sharply peaked — while the token is factually wrong.

Prompt: "The attention paper was published by"

Top token probabilities at next position:



Correct here — but replace with a less common paper and the model may hallucinate at $P=0.91$.

Why it is dangerous

Confident-sounding output passes casual human review. Users who trust the output without verification propagate the error. In legal, medical, and scientific contexts this is a production safety risk.

What confidence IS

Confidence is a statistical property of the training distribution. It measures how consistent the token is with patterns the model has seen — not how true it is in the world.

Build this reflex now and keep it: High probability is not equal to High accuracy. Confident output is not equal to Verified fact.

What Fixes Hallucination — and What Does Not

No single technique eliminates it. Each addresses a different part of the problem.

Prompt Engineering

Addresses: Reduces extrinsic hallucination by grounding the context

Clear instructions, relevant context, and asking the model to cite or abstain shifts probability mass toward faithful output.

Warning: Cannot fix factual gaps in training distribution. If the model never saw the correct information, no prompt can produce it.

Zero setup

Reliability: Low–Medium

Fine-Tuning and RLHF

Addresses: Reduces confident wrong answers; teaches abstention

RLHF reward models penalise hallucinated outputs, pushing probability mass away from confidently wrong tokens toward correct or hedged responses.

Warning: Reduces frequency but does not eliminate. Shifts the distribution — does not add new true knowledge.

High cost

Reliability: Medium improvement

RAG — Retrieval-Augmented Generation

Addresses: Directly addresses factual hallucination

Retrieved documents shift the probability distribution toward correct tokens. The model conditions on ground-truth evidence rather than training memory.

Warning: Retrieval quality limits output quality. If retrieval returns wrong documents, hallucination persists.

Moderate setup

Reliability: High for covered facts

Human-in-the-Loop Review

Addresses: Catches hallucinations before they reach end users

Expert review of model outputs before deployment. Especially important for high-stakes domains: legal, medical, financial, scientific.

Warning: Does not scale. Catches errors after generation. Requires domain expertise.

High ongoing cost

Reliability: High if expert

Nothing fully eliminates hallucination at present. This is an active open research problem across every major AI lab.

2.6

Common Misconceptions — Thoroughly Cleared

Three beliefs that must be corrected before advancing to the Transformer

Misconception 1 — "The model reads, thinks, then writes"

There is no two-phase process. No comprehension phase. No planning step.

MYTH: "The model reads the prompt, understands it, thinks about the answer, then writes it."

REALITY: The model begins generating the first output token immediately after the last prompt token.

The transformer processes the entire prompt in a single parallel forward pass. The output of this pass is one probability distribution. The model immediately samples the first output token. There is no pause, no planning, no internal scratchpad between the last prompt token and the first output token.

What looks like thinking

When a model writes 'Let me think step by step...' and then produces a correct answer, it is not thinking before answering. It is generating tokens that prime subsequent tokens toward more accurate continuations. Chain-of-thought is a generation strategy, not a pre-generation cognition phase.

Implication: Chain-of-thought prompting works not because the model thinks, but because each generated intermediate step shifts the probability distribution of the next step toward more accurate tokens.

Misconception 2 — "The model knows what it will say before it says it"

The fifth word does not exist before the fourth word has been committed.

MYTH: "The model has the full answer ready and is just displaying it word by word."

REALITY: The model is genuinely deciding one token at a time, committed only on what has already been generated.

Each token is committed before the next is chosen — there is no look-ahead:



The cascade effect

A different sampling draw at token 1 can cascade into a completely different completion by token 20. This is why the same prompt at the same temperature can produce different outputs across runs.

Practical implication

Run the same prompt multiple times at $T=1.0$ and you will get different outputs. Set $T=0$ for full determinism but at the cost of diversity and repetition risk.

The model has no answer stored. It has a weight matrix. Every generation is a fresh probabilistic draw.

Misconception 3 — "Confident output means correct output"

Confidence is statistical. Truth is external. The two are not the same.

MYTH: "If the model sounds certain, the output is probably correct."

REALITY: Confidence measures consistency with the training distribution — not alignment with external truth.

Anatomy of a confident hallucination — every stage looks normal:

Training data	Contained many confident-sounding sentences of the form X was published in Journal Y in year Z — even where specific values were occasionally wrong.
Logits	The logit for the wrong value is high because the sentence pattern matches the training distribution well, regardless of the specific value at that position.
Softmax output	$P(\text{wrong year} \mid \text{context}) = 0.91$. The distribution is sharply peaked. The model appears highly confident in the incorrect token.
Generated text	The paper was published in 1987. Stated without hedging, in the same fluent tone as true statements. Indistinguishable to a non-expert.

Permanent rule: Do not use fluency, tone, or apparent certainty as a proxy for accuracy. Always verify high-stakes claims from model output against authoritative external sources.

All Three Misconceptions at a Glance

Carry these corrections into every session from this point forward

M1

MYTH: "The model reads, thinks, then writes"

CORRECTION: No planning phase. The first output token is generated immediately. What looks like thinking is the generation of intermediate tokens that shift subsequent probabilities.

M2

MYTH: "The model knows what it will say before it says it"

CORRECTION: No stored answer. The model is a weight matrix. Every generation is a fresh probabilistic draw. Different sampling at token 1 cascades into completely different completions.

M3

MYTH: "Confident output means correct output"

CORRECTION: Confidence is statistical — consistency with training distribution, not truth. Confident hallucination passes casual review and is the most dangerous failure mode.

References & Further Reading

Foundational papers, tools, and resources for Topic 2

Foundational Papers

- [1] Holtzman et al. (2020). The Curious Case of Neural Text Degeneration. ICLR.
Defines repetition failure modes; introduces nucleus (top-p) sampling
- [2] Ji et al. (2023). Survey of Hallucination in Natural Language Generation. ACM.
Comprehensive taxonomy of hallucination types across NLG tasks
- [3] Maynez et al. (2020). On Faithfulness and Factuality in Abstractive Summarization.
Defines intrinsic vs extrinsic hallucination distinction
- [4] Ouyang et al. (2022). Training LMs to Follow Instructions with RLHF. (InstructGPT)
RLHF as a mitigation approach for hallucination and harmful output
- [5] Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP.
RAG as the primary technical fix for factual hallucination

Tools & Videos

- Video Andrej Karpathy — Let's build GPT from scratch
youtube.com/watch?v=kCc8FmEb1nY
- Video 3Blue1Brown — Transformers explained visually
youtube.com/3blue1brown
- Tool OpenAI API — stream=True streaming guide
platform.openai.com/docs/api-reference
- Tool Hugging Face — output_scores for logprobs
huggingface.co/docs/transformers
- Paper Liang et al. (2022). Holistic Evaluation of LMs (HELM)
crfm.stanford.edu/helm
- Blog Simon Willison — Hallucination is not a bug
simonwillison.net

Topic 2 Summary — Autoregressive Generation

Six essential takeaways

1

Autoregression = Topic 1 in a loop

No new math. The softmax mechanism from Topic 1 is applied repeatedly. Each forward pass = one new token committed to the context.

2

Three stopping conditions

EOS token (linguistic), stop sequences (structural), max_tokens (economic). finish_reason tells you which triggered. All are API-controllable.

3

Greedy decoding causes loops

High-probability phrases self-reinforce. Nucleus sampling (top-p) breaks the attractor by forcing distribution breadth across the nucleus.

4

Three types of hallucination

Intrinsic (contradicts context), extrinsic (unverifiable addition), factual (contradicts real world). Each has a different cause and different fix.

5

High confidence is not high accuracy

Softmax peaks on wrong tokens. Confident hallucination is indistinguishable from confident truth by fluency and tone alone. Always verify.

6

Three corrected misconceptions

No planning phase. No pre-stored answer. No reliability signal from confident tone. Carry all three corrections into every subsequent session.

Topics 1 & 2 — What Carries Forward

Building blocks that every subsequent topic depends on

Topic 1 Completed	Topic 2 Completed	Topic 3 Next	Topic 4 Next
✓ Token vocabulary ✓ Softmax + logits ✓ Temperature ✓ Sampling strategies ✓ Cross-entropy ✓ Perplexity	✓ Autoregressive loop ✓ Stopping conditions ✓ Repetition failure ✓ 3 hallucination types ✓ Confidence not equal truth ✓ 3 misconceptions cleared	○ Self-attention Q/K/V ○ Multi-head attention ○ Positional encoding ○ Causal masking ○ Full Transformer block ○ AR vs parallel encoding	○ ViT patch tokens ○ Image embeddings ○ VLM architecture ○ CLIP & contrastive ○ Shared embed space ○ Bridge to RAG/M5

Topic 2 Complete

Autoregressive Generation

Coming up next in this session:

Topic 3

Self-Attention Mechanism

Q, K, V — the library analogy, formula derivation step by step, causal masking, and autoregression vs parallel encoding.

Topic 4

Multimodal AI and Vision Transformers

Images as token sequences. ViT patch decomposition, image embeddings, VLM architecture, CLIP and shared embedding space.

A2 + A3

Assignments A2, A3, A4

Stream tokens (A2), attention visualization (A3), manual patch decomposition and ViT shape inspection (A4).

Assignment A2 issued · Submit before mentor session next week · Questions during the break